

Experiment-02: 8-Queens Problem using Python (Backtracking)

Aim

To develop a Python program that solves the **8-Queens Problem** using the **Backtracking Algorithm**, ensuring that no two queens attack each other on the chessboard.

Algorithm

1. Start with an empty 8×8 chessboard.
 2. Place a queen in the first column.
 3. Check whether the position is safe:
 - No queen exists in the same row.
 - No queen exists in the upper-left diagonal.
 - No queen exists in the lower-left diagonal.
 4. If the position is safe, place the queen.
 5. Move to the next column and repeat the process.
 6. If no safe position is available, backtrack by removing the previously placed queen.
 7. Continue until all 8 queens are placed successfully.
 8. Display the final chessboard configuration.
-

Python Code

```
N = 8
```

```
def print_board(board):  
    print("\nSolution:\n")  
    for i in range(N):  
        for j in range(N):  
            if board[i][j] == 1:  
                print("Q", end=" ")  
            else:
```

```
        print(".", end=" ")
    print()
```

```
def is_safe(board, row, col):
```

```
    # Check left side of current row
```

```
    for i in range(col):
```

```
        if board[row][i] == 1:
```

```
            return False
```

```
    # Check upper-left diagonal
```

```
    i, j = row, col
```

```
    while i >= 0 and j >= 0:
```

```
        if board[i][j] == 1:
```

```
            return False
```

```
        i -= 1
```

```
        j -= 1
```

```
    # Check lower-left diagonal
```

```
    i, j = row, col
```

```
    while i < N and j >= 0:
```

```
        if board[i][j] == 1:
```

```
            return False
```

```
        i += 1
```

```
        j -= 1
```

```
    return True
```

```
def solve(board, col):
```

```
if col >= N:
```

```
    return True
```

```
for row in range(N):
```

```
    if is_safe(board, row, col):
```

```
        board[row][col] = 1
```

```
        if solve(board, col + 1):
```

```
            return True
```

```
    # Backtracking
```

```
    board[row][col] = 0
```

```
return False
```

```
board = [[0 for _ in range(N)] for _ in range(N)]
```

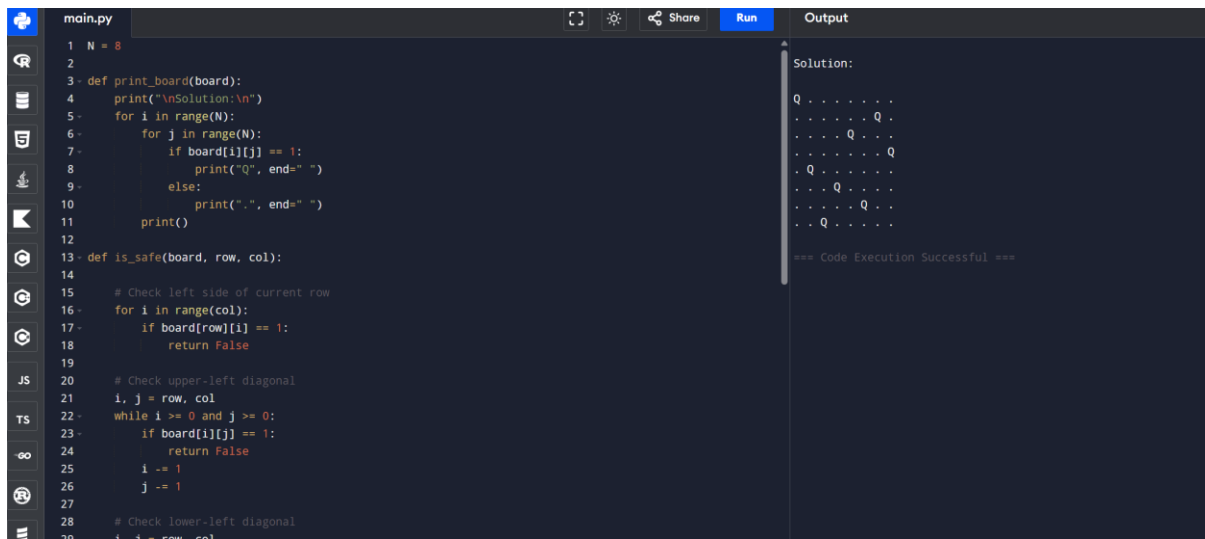
```
if solve(board, 0):
```

```
    print_board(board)
```

```
else:
```

```
    print("No solution exists.")
```

Output:



```
main.py
1 N = 8
2
3 def print_board(board):
4     print("\nSolution:\n")
5     for i in range(N):
6         for j in range(N):
7             if board[i][j] == 1:
8                 print("Q", end=" ")
9             else:
10                print(".", end=" ")
11        print()
12
13 def is_safe(board, row, col):
14
15     # Check left side of current row
16     for i in range(col):
17         if board[row][i] == 1:
18             return False
19
20     # Check upper-left diagonal
21     i, j = row, col
22     while i >= 0 and j >= 0:
23         if board[i][j] == 1:
24             return False
25         i -= 1
26         j -= 1
27
28     # Check lower-left diagonal
29     i, j = row, col
```

Solution:

```
Q . . . . .
. . . . . Q .
. . . Q . . .
. Q . . . . .
. . . Q . . .
. . . . Q . .
. . . . . Q .
. Q . . . . .

=== Code Execution Successful ===
```

Result

The **8-Queens Problem** was successfully solved using the **Backtracking Algorithm**. The program placed all eight queens on the chessboard such that no two queens attacked each other, demonstrating the effectiveness of backtracking in solving constraint satisfaction problems.